

Base de conocimiento vectorial para recuperación multilingüe

CODEFEST Ad Astra 2026 — Etapa 1 · Equipo **random_state = 42**

Davinson Arteaga · Juan Villegas · Daniela Castaño · Gian Mendoza

1. Objetivo y resumen ejecutivo

Se construyó un sistema de **recuperación densa** sobre el corpus multiformato y multilingüe de ADL (1.826 documentos, ES/EN/PT, tres fenómenos). Dadas 50 consultas en lenguaje natural, el sistema devuelve por cada una **3 documentos** y **10 fragmentos** de a lo sumo 250 palabras. No interviene ningún modelo generativo en ninguna etapa.

La arquitectura final es deliberadamente simple, porque es la que la evidencia sostuvo:

consulta → BGE-M3 → FAISS IndexFlatIP (top-100) → materialización M4 → normalización ≤ 250 palabras
→ 10 fragmentos → max-pooling documental → 3 documentos

Se evaluaron y **descartaron** un segundo encoder (GTE-multilingual), la fusión RRF y un reranker cross-encoder: ninguno mejoró las métricas primarias sobre nuestro conjunto de desarrollo. El sistema entregado usa un solo índice, lo que reduce almacenamiento, tiempo de consulta y superficie de fallo en la máquina del evaluador.

2. Corpus y preprocesamiento multiformato

El inventario reconciliado contra el índice de ADL arrojó **1.826 documentos**. La distribución real por formato resultó muy distinta de la sugerida por la especificación (que mencionaba PDF/HTML/MD): no hay ningún HTML, y JSON es el formato más frecuente por documento.

Formato	json	pdf	pbf	csv	jpg	xlsx	avif	txt
Documentos	954	759	73	26	8	4	1	1
Chunks	6.778	141.200	5.191	171.499	15	2.176	1	6

La extracción es **format-aware**: un extractor por familia de formato, cada uno produciendo el mismo contrato intermedio (**RawDoc**: identificador, ruta de origen, formato, fenómeno y bloques de texto). Los PDF se procesan con extracción de texto y OCR de respaldo; las tablas (CSV/XLSX) emiten cada fila como una secuencia **columna: valor**, de modo que cada valor conserva el nombre de su columna como contexto; los PBF emiten una unidad por **feature** geográfica. Los fallos de archivos individuales se registran y se saltan, nunca tumban el pipeline. La conservación es verificable: el manifiesto del chunking declara **0 palabras perdidas** entre la extracción y el artefacto indexado.

3. Estrategia de chunking

La política final, **format_aware_v2**, empaqueta unidades atómicas consecutivas del mismo documento hasta un objetivo de tamaño, sin partir jamás una unidad. La unidad atómica depende del formato: una **oración** en texto narrativo (segmentada con **pysbd**, nunca con **split(".")**), una **fila** en CSV/XLSX, una **feature** en PBF.

Parámetro	Valor	Parámetro	Valor
target_words	120	overlap_units	1
soft_min_words	72	cross_block_packing	true
max_words	250	chunks resultantes	326.866

El solapamiento se mide **en unidades**, no en palabras ni caracteres: es la única variante compatible con la regla de completitud lingüística. Se repite la última unidad del chunk anterior al inicio del siguiente, lo que eleva

la redundancia textual a 1,26× y evita que una idea quede partida entre dos chunks sin contexto.

Unidades atómicas sobredimensionadas. 1.547 chunks (0,47 %) superan las 250 palabras porque contienen una unidad indivisible que ya nace por encima del techo — típicamente tablas volcadas como texto corrido o PDF cuya extracción no produce fronteras de oración fiables. Truncarlas violaría la completitud lingüística, así que el chunker las emite enteras y es la capa de *salida* la que garantiza el límite de 250 palabras (§6). No es cierto, por tanto, que todos los chunks indexados quepan en 250 palabras; sí lo es que todos los fragmentos entregados lo hacen.

Trazabilidad: `config_fingerprint = f2c665528a008aa9`, `SHA256 = cc91d779...c3b87e4ea1c`. El manifest de entrega registra esta procedencia; el preflight de runtime verifica la coherencia estructural del índice, la metadata y la configuración del encoder, mientras que la identidad de los artefactos se certifica mediante hashes durante el empaquetado y la validación final.

4. Selección del encoder

Criterios, en orden de peso: multilingüe nativo ES/EN/PT (la recuperación cross-lingual es el caso central, no un extra), desempeño en recuperación densa, licencia permisiva, longitud de contexto y coste de inferencia. Se descartaron por licencia o por ser solo-inglés varios candidatos; se llevaron a medición dos finalistas, ambos Apache-2.0 y de familia encoder (BERT), nunca decoders.

Métrica	BGE-M3	GTE-multilingual	RRF (BGE+GTE)
ProxyNDCG@10	0,1294	0,0764	0,0995
F1@3	0,1958	0,0500	0,1125
Precision@3	0,1667	0,0417	0,0833
Recall@3	0,2708	0,0625	0,1875
Hit@3	0,5000	0,1250	0,2500
MRR	0,3057	0,1333	0,1800
Evidence Recall@100	0,4000	0,3333	0,4000

Decisión: un solo encoder, BGE-M3. GTE quedó por debajo en todas las métricas primarias. RRF no mejoró a BGE en ninguna de las dos: $\Delta\text{ProxyNDCG@10} = -0,0299$ y $\Delta\text{F1@3} = -0,0833$. A profundidad de operación (K=100) GTE no aportó ninguna unidad de evidencia exclusiva, aunque sí aportó una a profundidades menores — es decir, GTE encuentra antes cierta evidencia que BGE también acaba encontrando. La regla automática, en su lectura más conservadora, quedó *inconclusa* por esa evidencia exclusiva transitoria; el equipo resolvió mantener un único índice, porque el segundo no compra ninguna mejora medible y sí duplica almacenamiento y tiempo.

Configuración final: `BAAI/bge-m3`, revisión `5617a9f61b028005a4858fdac845db406aefb181`, 1.024 dimensiones, vectores L2-normalizados, contexto 8.192 tokens, sin prefijo de consulta, sin `trust_remote_code`. Fijar el commit exacto (y no `main`) es lo que impide que dos ejecuciones en fechas distintas usen pesos diferentes.

5. Índice FAISS y metadata

Se usa **IndexFlatIP** sobre vectores L2-normalizados: el producto interno es entonces el coseno *exacto*. Para el volumen del reto (326.866 vectores) la búsqueda exhaustiva es holgadamente viable y evita tener que justificar y medir la pérdida de exactitud de un índice aproximado (IVF, HNSW).

Propiedad	Valor	Propiedad	Valor
Tipo	IndexFlatIP	Vectores (ntotal)	326.866
Dimensión	1.024	Filas de metadata	326.866
Documentos	1.826	chunk_id únicos	326.866

La invariante crítica es que **la fila / de metadata.jsonl es el identificador interno / de FAISS**: la metadata se escribe en el mismo orden y en el mismo paso en que se hace `index.add()`, y nunca se reordena ni deduplica después. El runtime la verifica al cargar y aborta si no se cumple. Cada fila conserva los ocho campos obligatorios: `doc_id`, `chunk_id`, `fuentes`, `formato`, `fenomeno`, `posicion`, `num_tokens`, `texto`.

Se conservan `doc_id` y `fuentes` porque cumplen funciones distintas: `doc_id` es el identificador con el que se evalúa a nivel documental, y `fuentes` es la ruta relativa exacta del archivo original de ADL, útil para trazabilidad y auditoría. La salida reporta `doc_id`.

6. Recuperación y ranking final

Para cada consulta: se codifica con el **mismo** encoder y la misma normalización que construyeron el índice, se recuperan los **100** candidatos más similares, y sobre ese conjunto —sin volver a buscar nunca— se construye la respuesta.

Materialización M4 (`best_bge_similarity_adjacent_if_fits`). La unidad indexada y la entregada no tienen por qué coincidir: un chunk de 60 palabras entregado tal cual desperdicia ~190 palabras de cobertura. Para cada candidato se consideran sus vecinos inmediatos *anterior* y *siguiente* del mismo documento; entre los que producen una combinación de ≤ 250 palabras se elige el de mayor similitud BGE con la consulta, calculada reconstruyendo el vector del vecino desde el propio índice (el vecino no necesita estar en el top-100). Empate exacto → el anterior. La concatenación aplica un **merge por solapamiento exacto**: si el sufijo de unidades del izquierdo es literalmente idéntico al prefijo del derecho, se elimina la repetición —igualdad literal, nunca similitud aproximada. El presupuesto de 250 palabras se evalúa sobre el texto ya deduplicado. El `chunk_id` reportado sigue siendo el del candidato recuperado.

Sobre las 50 consultas oficiales, M4 eligió un vecino en el 92,9 % de los 5.000 candidatos procesados: **2.353** veces el siguiente, **2.293** el anterior y **354** veces ninguno.

Normalización de salida. Si el texto materializado supera las 250 palabras —lo que solo puede ocurrir con un chunk atómico sobredimensionado— se divide en subfragmentos respetando fronteras de oración reales. Si la unidad es indivisible y aun así no cabe, el candidato **se omite** (nunca se trunca, resume ni parafrasea) y se continúa con los siguientes candidatos *del mismo top-100*: jamás se recupera un candidato 101 para rellenar.

Agregación documental. Los 3 documentos salen del pool legal *completo* (los 100 candidatos), no solo de los 10 fragmentos mostrados, por max-pooling: la puntuación de un documento es la del mejor candidato entregable que lo respalda; empates por `doc_id` ascendente. Un candidato omitido no puede sostener un documento, de modo que ningún documento entra al top-3 gracias a contenido que no podría entregarse.

Sin modelos generativos. Ningún decoder ni LLM interviene en la indexación, la transformación de consultas, la recuperación, el reranking, la selección de fragmentos ni la materialización del texto. Todo el `text` entregado es una subsecuencia literal de unidades presentes en el índice. El uso de asistentes de código durante el desarrollo no forma parte del runtime.

7. Validación experimental

Las decisiones se tomaron con un protocolo de ablación: una variable por experimento, el mismo conjunto de desarrollo, y registro de configuración, métrica y delta antes de aceptar un cambio.

Alcance y honestidad del conjunto de desarrollo. Las métricas de la §4 provienen de un conjunto interno de **9 consultas (8 evaluables) y 15 unidades de evidencia** anotadas por el equipo, evaluadas con un proxy de NDCG basado en cobertura textual 5-grama (umbral 0,95). Es un conjunto pequeño y sesgado hacia PDF: sirve para separar alternativas con diferencias grandes —como BGE frente a GTE, donde la brecha es de más del doble en las métricas primarias— pero **no autoriza a declarar significancia estadística** ni a afinar diferencias pequeñas. Lo declaramos explícitamente en vez de presentar los números como definitivos.

Chunking. Se barrieron granularidad y solapamiento. Bajar el objetivo de 200 a 120 palabras subió el techo de representación de la evidencia medido sobre el conjunto de desarrollo, y activar un solapamiento de una unidad resultó auditable y sin pérdida de contenido; ambas se promovieron a `format_aware_v2`.

Materialización. Se compararon cinco políticas (desde devolver el chunk crudo hasta usar la similitud del vecino) contra un oráculo que elegía el vecino mirando el texto de referencia. M4 fue la política sin acceso a las etiquetas que más se acercó a ese techo, y es la que se entrega.

Reranker. Se evaluó un cross-encoder (permitido: no es un decoder) reordenando el pool recuperado. No produjo una mejora metodológicamente defendible sobre el conjunto disponible, así que **se descartó** y no forma parte del pipeline final. Tampoco se implementó el bonus de grafo de conocimiento: la prioridad fue una entrega válida y reproducible.

Frontera con las etiquetas. El runtime de entrega es *gold-free por construcción*: no importa —ni siquiera de forma indirecta— ningún módulo de evaluación, devset o métrica. Es una frontera física verificada por pruebas automáticas que importan el pipeline en un proceso limpio y comprueban qué módulos quedaron cargados.

8. Reproducibilidad, configuración final y limitaciones

La entrega se ejecuta con `python generador.py` desde su propia carpeta, sin argumentos, variables de entorno ni edición de código. Requiere **Python 3.9.5 o superior** y **CPU**; no regenera embeddings del corpus: carga el índice, codifica las 50 consultas y busca. Verificado en un entorno limpio de CPython 3.9.25 instalando únicamente `requirements.txt`.

Elemento	Valor
Encoder	BAAI/bge-m3 @ 5617a9f61b028005a4858fdac845db406aefb181
Índice	IndexFlatIP · 1.024 dim · 326.866 vectores
SHA256 index.faiss	c5741cd0344cb644ea06b04f8ac14f5d4a8c7965cf6c4a067e0a21ccc8127a53
SHA256 metadata.jsonl	b20f70452fabf3ea7562c47a7909c77f07c6fdb5ff45d247e45c7b7d657c91d0
candidate_k	100
Salida	3 documentos + 10 fragmentos ≤250 palabras

La ejecución es **determinista**: mismas consultas, mismo índice y misma revisión del encoder producen un `resultados.jsonl` byte a byte idéntico, comprobado repitiendo la generación y comparando SHA256. No se usa aleatoriedad en ninguna etapa. Sobre las 50 consultas oficiales, los 500 fragmentos entregados tienen entre 103 y **249** palabras (mediana 184).

Limitaciones. (i) El paquete no incluye los pesos del encoder, así que la primera ejecución descarga BGE-M3 desde HuggingFace fijando la revisión exacta; el runtime resuelve primero un checkpoint local si se le proporciona, de modo que la ejecución puede hacerse totalmente offline. (ii) 13 candidatos, repartidos en 10 de las 50 consultas, resultaron unidades atómicas indivisibles de más de 250 palabras y se omitieron; el pool legal nunca bajó de 98 candidatos, así que las 50 consultas entregan 10 fragmentos y 3 documentos completos. (iii) No se entrega el bonus de grafo de conocimiento. (iv) Las métricas internas provienen de un conjunto de desarrollo pequeño, como se detalla en la §7.